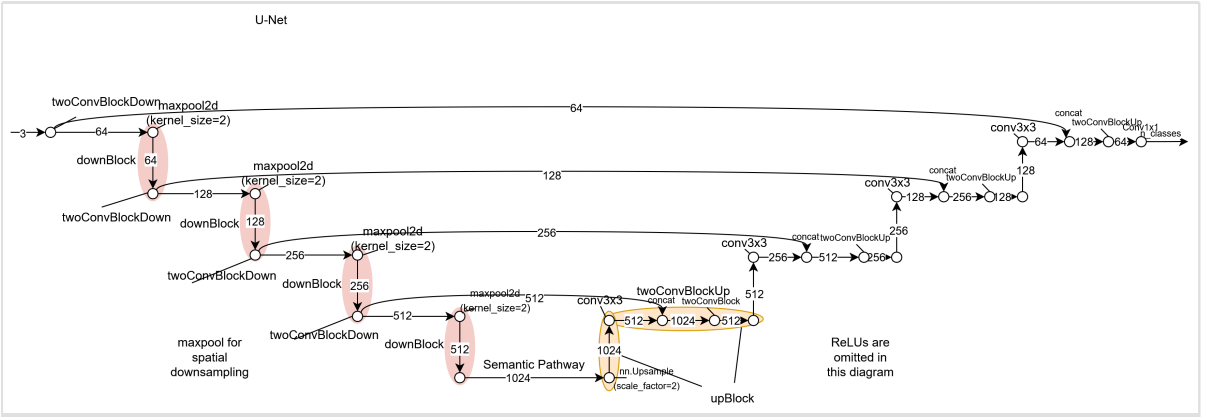
Dense_Prediction

2D

1. CNN-based
    1. In computer vision pixelwise dense prediction is the task of predicting a label for each pixel in the image https://arxiv.org/abs/1611.09288.
        1. Semantic segmentation: classify per pixel
        2. Depth estimation: regress per pixel.
        3. Instance segmentation: classify per pixel per object.
        4. Panoptic segmentation: handles both stuff (semantic segmentation) and things (instance segmentation).
            1. Assign (category, instance_id) pair to each pixel in image.
            2. Instance label ignored for "stuff" categories.
    2. FCN for semantic segmentation
        1. Why not stack convolutions? suppose 200 3x3 filters/layer, H=W=400. Storage/layer/image: 200 *400* 400 * 4 bytes = 122MB, 122MB*100 layers, batch size of 20 = 238 GB of memory. [2]
        2. Key idea: encoder-decoder, first downsample towards middle of network, then upsample from middle,
        3. how do we downsample? convolutions, pooling.
        4. Putting it together: conv+pooling downsample/compress/encode, Transpose convs/unpoolings upsample/uncompress/decode.
    3. U-Net for semantic segmentation and depth estimation
        1. 

UNet|
        2. Code: unet.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F


def conv3x3(input_dim, output_dim, stride=1, padding=0, bias=True):
    """3x3 convolution with padding"""
    return nn.Conv2d(
        input_dim, output_dim, kernel_size=3, stride=stride, padding=padding, bias=bias
    )


class twoConvBlock(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        # todo
        # initialize the block
        self.block = nn.Sequential(
            conv3x3(input_dim, output_dim, stride=1, padding=0, bias=False),
            nn.ReLU(inplace=True),
            conv3x3(output_dim, output_dim, stride=1, padding=0, bias=False),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        # todo
        # implement the forward path
        return self.block(x)
```

```python
class twoConvBlockDown(twoConvBlock):
    expansion = 2

    def __init__(self, input_dim):
        output_dim = input_dim * twoConvBlockDown.expansion
        super().__init__(input_dim, output_dim)
        # self.block = nn.Sequential(
        #     conv3x3(input_dim, input_dim * twoConvBlockM.expansion,
        #             stride=1, padding=0, bias=False),
        #     nn.ReLU(inplace=True),
        #     conv3x3(input_dim, input_dim * twoConvBlockM.expansion,
        #             stride=1, padding=0, bias=False),
        #     nn.ReLU(inplace=True),
        # )


class twoConvBlockUp(twoConvBlock):
    def __init__(self, input_dim):
        output_dim = input_dim // 2
        super().__init__(input_dim, output_dim)

    def forward(self, x1, x2):
        # concat
        # x1: input from skip connection
        # x2: input from upsample
        # print('x1', x1.shape)
        # print('x2', x2.shape)
        diff_y = x1.shape[2] - x2.shape[2]
        diff_x = x1.shape[3] - x2.shape[3]
        x2 = torch.nn.functional.pad(
            x2, [diff_x // 2, diff_x - diff_x // 2, diff_y // 2, diff_y - diff_y // 2]
        )
        x = torch.cat([x1, x2], dim=1)
        return self.block(x)


class downBlock(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.block = nn.Sequential(
            nn.MaxPool2d(kernel_size=2),
            twoConvBlockDown(input_dim),
        )

    def forward(self, x):
        return self.block(x)


class upBlock(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.upsample = nn.Sequential(
            nn.Upsample(scale_factor=2),
            conv3x3(input_dim, input_dim // 2),
        )
        self.twoconvblock = twoConvBlockUp(input_dim)

    def forward(self, x_skip, x_lower):
        # print('x_lower', x_lower.shape)
        x_upsample = self.upsample(x_lower)
        # print('x_upsample', x_upsample.shape)
        # print('x_skip', x_skip.shape)
        x1 = self.twoconvblock(x_skip, x_upsample)
        # print('x1.shape', x1.shape)
        return x1


class downStep(nn.Module):
    def __init__(self):
        super().__init__()
        # todo
        # initialize the down path
        layers = [twoConvBlock(1, 64)]
        in_planes = [64, 128, 256, 512]
        self.hooks = {}
        for i in range(len(in_planes)):
            layers.append(downBlock(in_planes[i]))

        for i in range(len(layers) - 1):
            layers[i].register_forward_hook(self.forward_hooks(f"down.{i}"))

        self.down = nn.Sequential(*layers)

    def forward_hooks(self, name):
        # Helper function to store features from the hook
        def hook(model, input, output):
            self.hooks[name] = output

        return hook
```

```
    def forward(self, x):
        # todo
        # implement the forward path
        # if len(self.hooks) > 0:
        #     print('self.hooks', self.hooks['down.0'].shape)
        #     print('self.hooks', self.hooks['down.1'].shape)
        #     print('self.hooks', self.hooks['down.2'].shape)
        #     print('self.hooks', self.hooks['down.3'].shape)
        return self.down(x), self.hooks


class upStep(nn.Module):
    def __init__(self):
        super().__init__()
        # todo
        # initialize the up path
        self.layers = nn.ModuleList([])
        in_planes = [1024, 512, 256, 128]
        for i in range(len(in_planes)):
            self.layers.append(upBlock(in_planes[i]))

    def forward(self, x, skip_connections):
        # todo
        # implement the forward path
        # x: feature tensor of semantic pathway
        for i in range(len(self.layers)):
            x_skip = skip_connections[f"down.{len(self.layers)-i-1}"]
            x = self.layers[i](x_skip, x)

        return x


class UNet(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        # todo
        # initialize the complete model
        self.down = downStep()
        self.up = upStep()
        self.final_block = nn.Sequential(
            nn.Conv2d(64, n_classes, kernel_size=1, stride=1, padding=0),
            # nn.Sigmoid(),
        )

    def forward(self, x):
        # todo
        # implement the forward path
        # Semantic pathway
        x = F.pad(x, [2, 2, 2, 2], "constant")
        x_semantic, skip_connections = self.down(x)
        x = self.up(x_semantic, skip_connections)
        out = F.pad(x, [2, 2, 2, 2], "constant")
        return self.final_block(out)


if __name__ == "__main__":
    model = UNet(n_classes=2).cuda()
    print(model)
    # x = torch.rand(4, 3, 256, 256).cuda()
    x = torch.rand(4, 1, 256, 256).cuda()
    out = model(x)
    print(out.shape)
```

4. Mask R-CNN for instance segmentation
    1. mask R-CNN is faster R-CNN with convolution branch.
    2. Boxes first paradigm:
        1. Run detector (Faster-RCNN).
        2. Produce segmentation relative to each predicted box.
    3. Mask R-CNN combines both steps into an end-to-end trainable model.
    4. Mask R-CNN is an extension of Faster R-CNN with additional branch for predicting segmentation mask on each Region of Interest (RoI). In the second stage of Faster R-CNN, RoI pool is replaced by RoIAlign which helps to preserve spatial information which gets misaligned in case of RoI pool. RoIAlign uses binary interpolation to create a feature map that is of fixed size for e.g. 7×7 [3].
    5. The output from RoIAlign layer is then fed into Mask head, which consists of two convolution layers. It generates mask for each RoI, thus segmenting an image in pixel-to-pixel manner [3].

2. Transformer
    1. Dense Prediction Transformer (DPT) for semantic segmentation and depth estimation
    2. SETR (semantic segmentation transformer)

3D

1. semantic segmentation:
    1. Point cloud: PointNet
        1. point cloud shape: $(n, 3)$ for n: number of points, 3 for x, y, and z.
        2. There are 2 important parts in pointnet:
            1. Apply MLP from 3 to 64 dimensions
                1. Then from 64 to 64 dimensions.
                2. get $n \times 64$ feature.
                3. only extract feature from each point, but not global.
            2. T-Net (to be invariant to permutations), to extract some global features from a set of point cloud features or point cloud coordinates:
                1. We should have a function that is symmetric to each point
                2. If $g(h(input)) = \gamma(g(h(x_1), h(x_2), \ldots, h(x_n)))$, then although we permute the input, the output will not change if the function is symmetric
                3. In this paper max_pooling is the best choice for the symmetric function
                4. input->MLP->max->MLP, h=MLP, g=max, $\gamma = \mathrm{MLP}$.
        3. T-Net (Input alignment by transformer network (to be invariant to any kind of rotation and transformation). T-Net is actually a simpler version of Spatial Transformer Network by Jaderberg et al.
            1. After the points goes through the T-Net, we can get some global features. We can use the global features to predict some inverse rotation matrix to make sure our input data can be inversely rotated into our canonical our original camera pose. This is the main motivation of input alignment by T-Net output.
            2. the transformation is just matrix multiplication.
            3. $\mathrm{matmul}(n \times 3, 3 \times 3)$. The 3×3 transformation matrix is output of T-Net.
        4. Then apply 3 layer MLP for local embedding, then get $n \times 1024$
        5. Apply max pooling to the $n \times 1024$ local embedding to get global feature, global feature size is 1×1024.
        6. After getting local features, use extra MLPs to perform classification that outputs k scores by applying a softmax and cross-entropy to supervise the whole network.
        7. T-Net is actually a Spatial Transformer Network (STN) or specifically STN3d and STN64d, the differences with the original STN are
            1. There is no grid generator needed since the point cloud itself is already like coordinates of a grid, we can directly apply a direct transformation H to transform the points i.e. $x' = Hx$. Related: Bilinear_Interpolation used in original STN.
            2. Instead of predicting the inverse transformation matrix like in the original STN paper, the STN in pointnet is predicting the direct transformation matrix to directly transform the point cloud by predicting only the 9 dof, 3x3 A matrix in the full affine transformation in 3D which overall has 12 dof, 9+3, 3 for translation vector t. Related: 3D_vision for affine transformation in 3D, Homography for the direct vs inverse transformation.
        8. pointnet.pytorch/pointnet/model.py

```
from __future__ import print_function
import torch
import torch.nn as nn
import torch.nn.parallel
import torch.utils.data
from torch.autograd import Variable
import numpy as np
import torch.nn.functional as F


class STN3d(nn.Module):
    def __init__(self):
        super(STN3d, self).__init__()
        self.conv1 = torch.nn.Conv1d(3, 64, 1)
        self.conv2 = torch.nn.Conv1d(64, 128, 1)
        self.conv3 = torch.nn.Conv1d(128, 1024, 1)
        self.fc1 = nn.Linear(1024, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 9)
        self.relu = nn.ReLU()

        self.bn1 = nn.BatchNorm1d(64)
        self.bn2 = nn.BatchNorm1d(128)
        self.bn3 = nn.BatchNorm1d(1024)
        self.bn4 = nn.BatchNorm1d(512)
        self.bn5 = nn.BatchNorm1d(256)


    def forward(self, x):
        batchsize = x.size()[0]
        x = F.relu(self.bn1(self.conv1(x)))
```

```
        x = F.relu(self.bn2(self.conv2(x)))
        x = F.relu(self.bn3(self.conv3(x)))
        x = torch.max(x, 2, keepdim=True)[0]
        x = x.view(-1, 1024)

        x = F.relu(self.bn4(self.fc1(x)))
        x = F.relu(self.bn5(self.fc2(x)))
        x = self.fc3(x)

        iden =
Variable(torch.from_numpy(np.array([1,0,0,0,1,0,0,0,1]).astype(np.float32))).view(1,9).repeat(batchsize,1)
        if x.is_cuda:
            iden = iden.cuda()
        x = x + iden
        x = x.view(-1, 3, 3)
        return x


class STNkd(nn.Module):
    def __init__(self, k=64):
        super(STNkd, self).__init__()
        self.conv1 = torch.nn.Conv1d(k, 64, 1)
        self.conv2 = torch.nn.Conv1d(64, 128, 1)
        self.conv3 = torch.nn.Conv1d(128, 1024, 1)
        self.fc1 = nn.Linear(1024, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, k*k)
        self.relu = nn.ReLU()

        self.bn1 = nn.BatchNorm1d(64)
        self.bn2 = nn.BatchNorm1d(128)
        self.bn3 = nn.BatchNorm1d(1024)
        self.bn4 = nn.BatchNorm1d(512)
        self.bn5 = nn.BatchNorm1d(256)

        self.k = k

    def forward(self, x):
        batchsize = x.size()[0]
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        x = F.relu(self.bn3(self.conv3(x)))
        x = torch.max(x, 2, keepdim=True)[0]
        x = x.view(-1, 1024)

        x = F.relu(self.bn4(self.fc1(x)))
        x = F.relu(self.bn5(self.fc2(x)))
        x = self.fc3(x)

        iden =
Variable(torch.from_numpy(np.eye(self.k).flatten().astype(np.float32))).view(1,self.k*self.k).repeat(batchsi
ze,1)
        if x.is_cuda:
            iden = iden.cuda()
        x = x + iden
        x = x.view(-1, self.k, self.k)
        return x
```

    2. Voxel: 3D U-Net
2. instance segmentation: SoftGroup

References:

1. CSEP 576: Dense prediction, Udub, Jonathan Huang, 2020.
2. CSEP 576: Deep learning in 3D, Udub, Vita Ablavsky, 2021.
3. How Mask R-CNN Works?
4. https://github.com/fxia22/pointnet.pytorch/blob/master/pointnet/model.py